



Relazione Progetto PMO

Applicativo di un videogioco di simulazione di vita.

Nasrine Aboufaris

n.aboufaris@campus.uniurb.it
321885

Nicole Fabbri

n.fabbri5@campus.uniurb.it
321119

Alessia Giuseppetti

a.giuseppetti@campus.uniurb.it
322984

Alice Neri

a.neri14@campus.uniurb.it
32165

Indice

1. Analisi	4
1.1. Requisiti	5
1.1.1. Requisiti minimi	5
1.1.2. Requisiti opzionali	5
1.2. Modello del dominio	5
2. Design	7
2.1. Architettura	7
2.1.1. Model	7
2.1.2. View	7
2.1.3. Controller	8
2.2. Design dettagliato	8
2.2.1. Aboufaris	8
★ Room	8
★ House	8
★ GameItem e ItemFactory	9
★ Requirement	10
★ Stats	10
2.2.2. Fabbri	11
★ MainCharacter	11
★ Outfit e Hair	11
★ FoodType	12
★ Controller	13
2.2.3. Giuseppetti	14
★ NPC	14
★ Mum, Dad and Brother	14
★ Quest	15
★ CompletionCondition	15
★ ItemUsageCondition	16
★ ItemDeliveryCondition	16
★ LevelRequirement	16
★ QuestSystem	16
2.2.4. Neri	18
★ ActionResult	18
★ Refrigerator	18
★ DropItemAction e PickItemAction	18
★ Inventory	18
★ View	20
3. Sviluppo	22

3.1. Testing automatizzato	22
3.1.1. HouseTest	22
3.1.2. GameItemTest	23
3.1.3. ItemFactoryKitchenTest	23
3.1.4. ItemFactoryBedroomTest	24
3.1.5. RefrigeratorTest	24
3.1.6. StatsTest	25
3.2. Metodologia di lavoro	26
Strumenti utilizzati	26
3.3. Sorgenti	26

1. Analisi

L'obiettivo del progetto è la realizzazione di un'applicazione che simuli il funzionamento di un life simulator, ispirato alla console My Life.

Il sistema è composto da:

- ★ Un personaggio principale, controllato dall'utente.
- ★ Tre NPC, ciascuno responsabile dell'assegnazione di una missione.
- ★ Sei stanze, ognuna dotata di oggetti diversi e interattivi.
- ★ Un sistema di bisogni del personaggio, come energia, fame e igiene.
- ★ Un sistema di gestione delle missioni che il personaggio deve svolgere.

Il flusso del sistema prevede che il giocatore crei il proprio personaggio e successivamente viene situato nella stanza iniziale predefinita: la camera da letto. L'unica altra stanza accessibile nella fase iniziale è la cucina mentre le altre hanno un requisito d'accesso determinato dal livello del personaggio. L'ingresso in una stanza permette l'interazione con l'eventuale NPC associato e l'utilizzo degli oggetti presenti. L'esecuzione delle azioni tramite gli oggetti ha un effetto sui bisogni del personaggio e, livelli troppo bassi di determinati bisogni, bloccano la possibilità di svolgere azioni diverse da quelle propense ad alzare il livello del bisogno critico. Le missioni, assegnate automaticamente dagli NPC al momento dell'entrata nella loro stanza prefissata, si considerano completate alla prima interazione con l'NPC in cui tutti i requisiti risultano soddisfatti.

La partita può concludersi in due esiti:

- ★ **Vittoria**, quando vengono completate tutte le missioni degli NPC.
- ★ **Sconfitta**, se uno dei bisogni del personaggio scende a zero prima del completamento.

1.1. Requisiti

1.1.1. Requisiti minimi

- ★ Permettere all'utente di inserire il nome del personaggio e personalizzarlo all'avvio.
- ★ Spostamento del personaggio tra stanze adiacenti.
- ★ Implementazione di tre NPC e una missione per ciascuno di essi.
- ★ Interazione con gli NPC per svolgere le missioni.
- ★ Display dei bisogni del personaggio.
- ★ Interazione esclusiva con gli oggetti della stanza in cui è situato il personaggio.
- ★ Sezione delle missioni disponibili e il loro stato.
- ★ Inventario del personaggio capace di contenere un numero finito di oggetti.

1.1.2. Requisiti opzionali

- ★ Livello di affinità con l'NPC che aumenta una volta completata la sua missione.
- ★ Oggetti speciali sbloccabili in situazioni specifiche.
- ★ Spostamento di oggetti tra una stanza e l'altra.

1.2. Modello del dominio

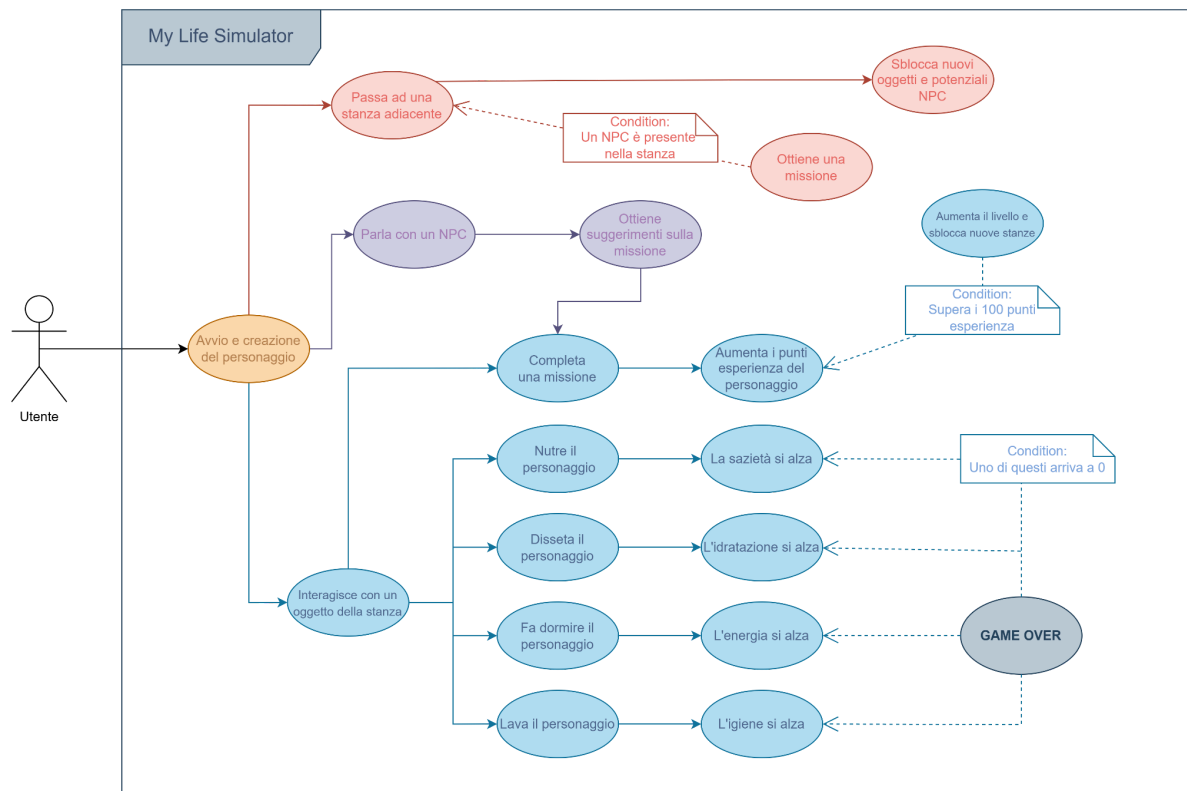
L'entità principale della simulazione è **MainCharacter** che rappresenta il personaggio principale del gioco. Attraverso essa l'utente può interagire con le altre entità maggiori: NPC, House e Item.

Gli **NPC** sono tre istanze che svolgono il medesimo ruolo: assegnare la propria **Quest** al MainCharacter e, quando l'Affinity con il personaggio supera una determinata soglia, possono collaborare durante l'esecuzione di specifici obiettivi.

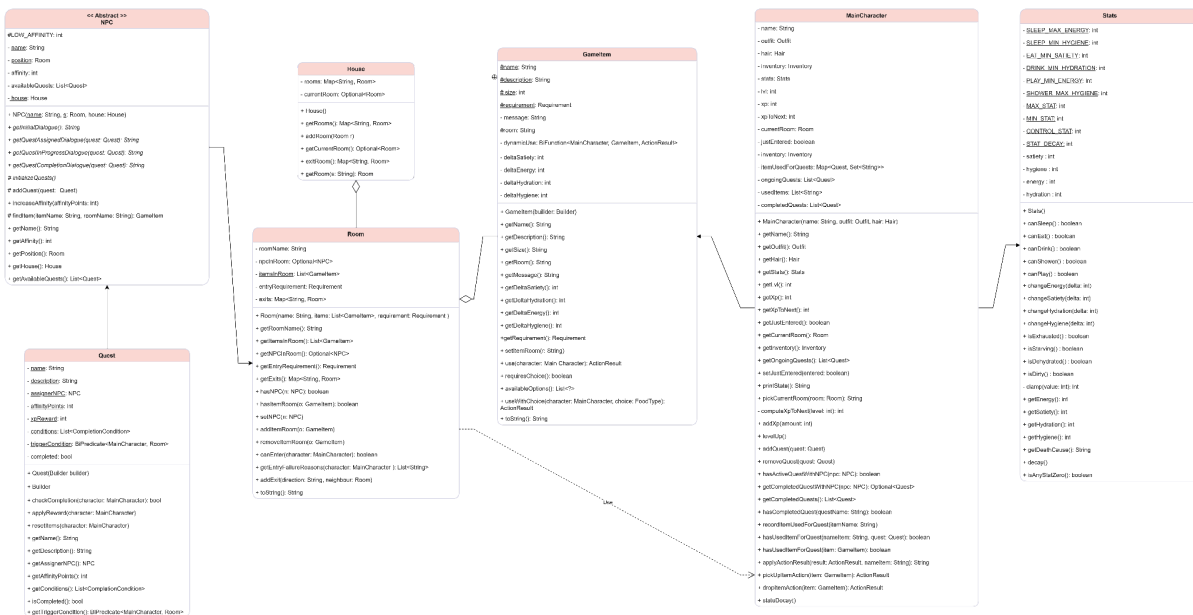
La **House** è articolata in più **Room**, ciascuna contenente **Item** diversi con i quali il MainCharacter può interagire per svolgere le Quest o per alterare i propri bisogni. Entrare in una Room abilita le azioni sugli Item presenti e, quando previsto, l'assegnazione di una missione da parte dell'NPC associato e le interazioni con esso.

MainCharacter mantiene una scheda di **Stats**, i cui valori vengono modificati dalle azioni sugli Item. Valori troppo bassi di determinati bisogni possono bloccare determinate azioni del personaggio, come esplicitato nei **Requirements**. Al livello 0 di uno solo degli Stats si incontra la schermata di Game Over e il simulatore termina.

Diagramma dei Casi d'Uso



Modello del dominio



2. Design

2.1. Architettura

L'architettura del sistema è basata sul pattern **Model-View-Controller**, che permette di separare la logica del gioco, contenuta all'interno del Model, dalla rappresentazione su schermo (View) e dal controllo degli input dell'utente (Controller). In questo modo si favorisce la modularità, manutenibilità e testabilità del codice.

Il **Model** rappresenta lo stato e la logica di dominio del gioco. Al suo interno ci sono le entità principali come **MainCharacter**, **NPC**, **Room**, **House**, **GameItem**, **Stats**, **QuestSystem** e le regole di interazione tra di esse. Il Model è stato progettato in modo che possa essere completamente indipendente dalla View e dal Controller.

La **View** presenta lo stato del Model e raccoglie gli input dell'utente senza conoscere i dati interni del dominio.

Infine, il **Controller** traduce gli eventi della View in azioni su Model e propaga alla View gli aggiornamenti rilevanti.

2.1.1. Model

La classe Model si occupa dello stato del gioco e delle regole del dominio. Le componenti principali sono:

- ★ **Action** che si occupa degli effetti delle interazioni del personaggio con gli oggetti sui propri bisogni e sulle missioni.
- ★ **Character** che contiene la logica dietro il personaggio principale, i suoi attributi e statistiche e la gestione degli NPC.
- ★ **Quest** che si occupa dell'avanzamento e delle condizioni delle missioni.
- ★ **Requirement** che contiene le condizioni bloccanti per determinate azioni sugli oggetti e sulle stanze.
- ★ **World** che gestisce la costruzione della casa, delle stanze e degli oggetti.

2.1.2. View

La View comunica strettamente con il **Controller** seguendo il pattern MVC: non modifica mai direttamente i dati del Model, ma inoltra ogni interazione dell'utente (click sui pulsanti, scelte dai menu) ai metodi del Controller (es. **handleUseItem**, **handleMove**). Inoltre, la classe gestisce finestre di dialogo modali (**JOptionPane**) per le interazioni complesse, come la selezione specifica del tipo di cibo (**FoodType**) quando si interagisce con il frigorifero, soddisfacendo i requisiti di interazione avanzata definiti nell'analisi.

2.1.3. Controller

La classe **Controller** svolge il ruolo di intermediario logico e operativo tra le due classi precedenti: traduce le interazioni dell'utente con l'interfaccia grafica (come lo spostamento tra le stanze o l'utilizzo degli oggetti) in comandi diretti verso le entità del dominio, per poi aggiornare la visualizzazione con i cambiamenti effettuate.

Una mansione importante svolta dal controller è la gestione del **tempo di gioco**: viene utilizzato un **ScheduledExecutorService** per implementare il loop temporale che attiva il decadimento delle statistiche del personaggio ogni 5 secondi, garantendo la dinamicità del gioco.

Infine, la classe gestisce la logica di **Game Over**, monitorando costantemente le condizioni vitali del personaggio e bloccando i comandi della View qualora uno di questi raggiunga lo zero.

2.2. Design dettagliato

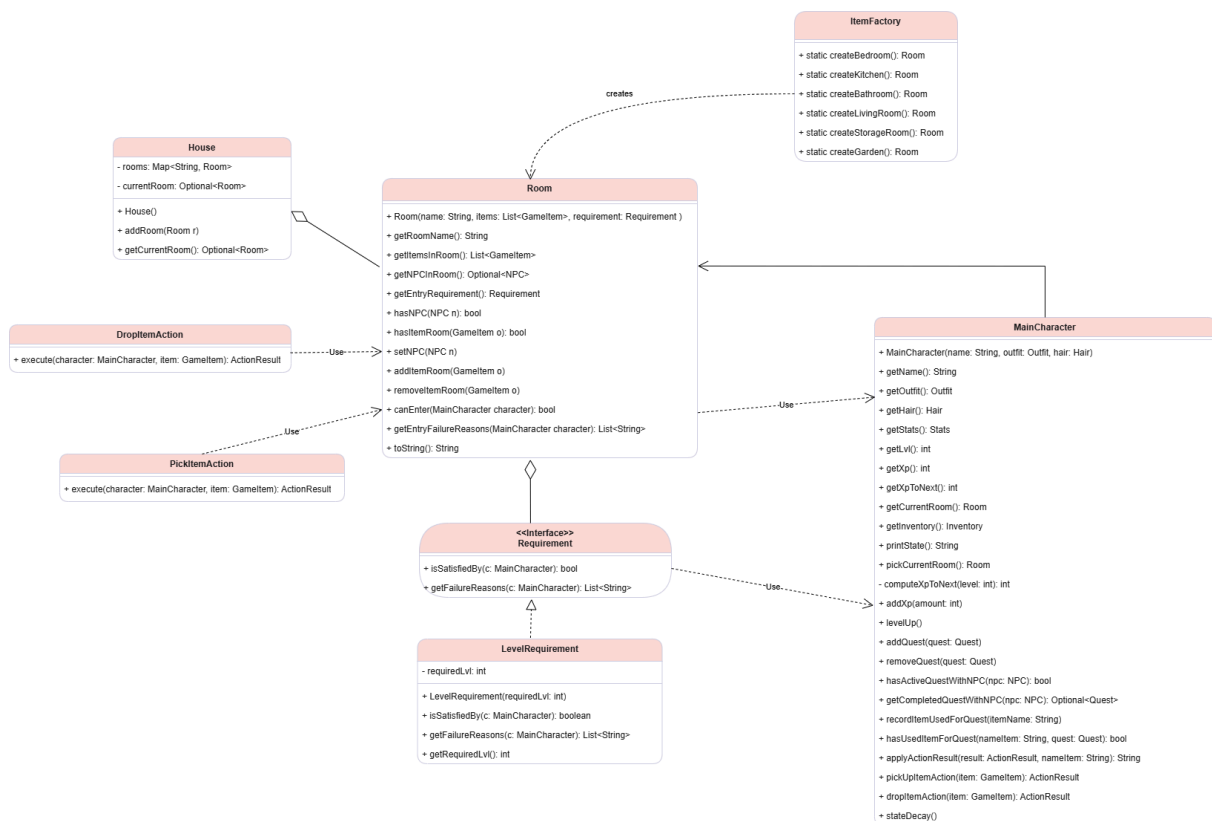
2.2.1. Aboufaris

★ Room

L'entità **Room** modella una singola stanza della casa e incapsula il nome, una collezione di **Gameltem** interattivi ed un riferimento opzionale all'**NPC** presente. All'interno di Room non è garantita la presenza di un **NPC**, per cui si è fatto uso di **Optional**, in modo da gestire i casi di null in maniera più sicura e pulita. La classe offre diversi metodi che permettono di rimuovere o aggiungere **Gameltem**, o anche di verificarne la presenza. L'accesso a Room viene gestito mediante l'interfaccia **Requirement**, così che i criteri possano essere estendibili senza modificare Room.

★ House

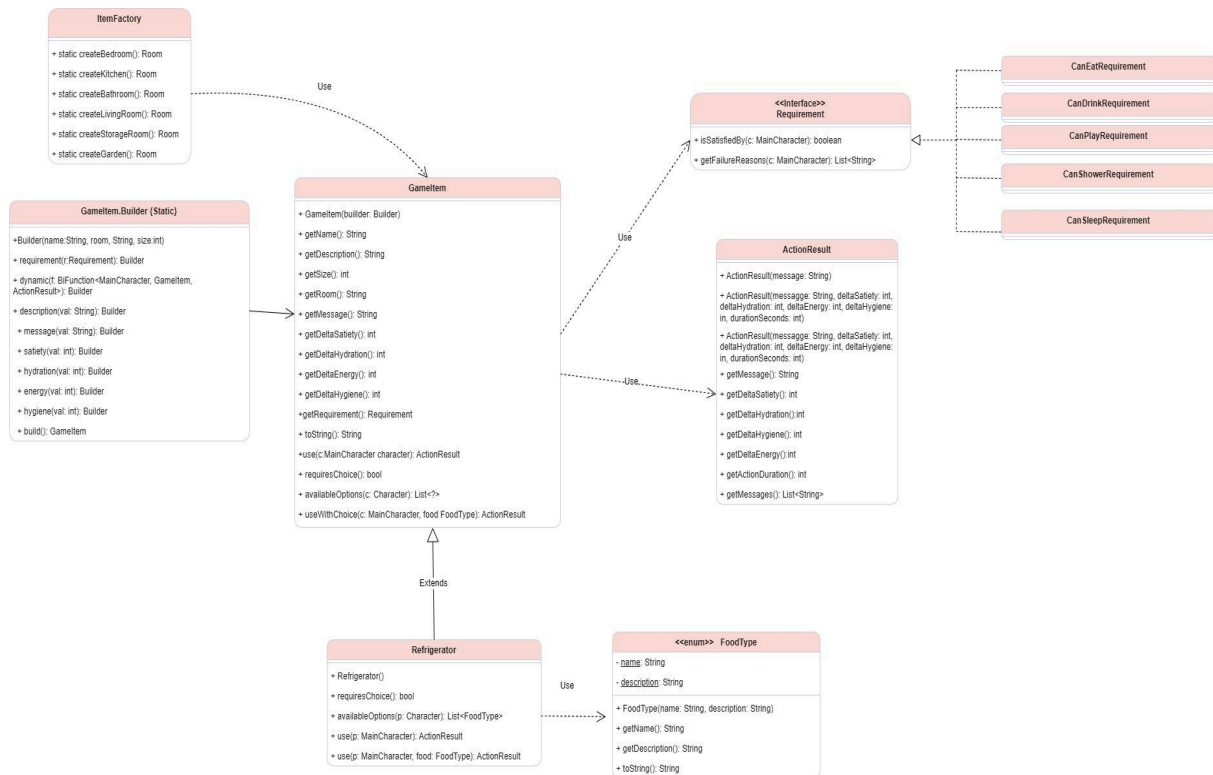
L'entità **House** aggrega le stanze tramite una mappa `Map<String, Room>` basandosi sull'accesso per nome e mantiene un `Optional<Room> currentRoom` per rappresentare la **Room** corrente in cui si trova il **MainCharacter**. Si è deciso di fare uso dell'**Optional** in quanto all'avvio del gioco, l'utente non si trova all'interno di una **Room** specifica. Metodi come `enterRoom()` ed `exitRoom()` gestiscono l'entrata/uscita dalla **Room**.



★ GamelItem e ItemFactory

L'entità **GamelItem** rappresenta un oggetto interattivo ed è costruita con un Builder interno per gestire in modo leggibile e sicuro le numerose proprietà opzionali. Si tratta di una classe che detiene molti parametri opzionali, per cui la scelta del Builder Pattern era la migliore per leggibilità e manutenzione del codice. Ogni **item** ha nome, descrizione, **Requirement** d'uso e un'interazione con il personaggio che modifica le **Stats**. Viene offerta anche una funzione `dynamic(MainCharacter, GameItem) -> ActionResult` la cui presenza calcola effetti e durata di un'azione in modo contestuale. La funzione `use(MainCharacter c)`, che è la funzione principale con cui il **MainCharacter** interagisce con il **GamelItem**, verifica i vincoli e, se presente applica la funzione dinamica, in ogni caso restituisce un **ActionResult**.

La creazione di **Room** e **GamelItem** è centralizzata in **ItemFactory**, che compone **Room** con la lista di **GamelItem** e il relativo **LevelRequirement**, e ospita le costanti di bilanciamento (durate, costi/benefici) mantenendo la logica vicino alla definizione degli oggetti. Per elementi come Letto, Divano, o Doccia, il Builder accetta una lambda di uso dinamico, permettendo di variare soglie o effetti.



★ Requirement

L'interfaccia **Requirement** applica il pattern Strategy ai vincoli d'uso dei **GametItem** e d'accesso alle **Room**. Definisce due metodi: `isSatisfiedBy(MainCharacter c)` e `getFailureReasons(MainCharacter c)`. Il primo interroga le statistiche del personaggio per controllare se un'azione può essere svolta o meno e, infine, restituisce un boolean. Il secondo serve a spiegare le ragioni di fallimento di un'azione e lo fa restituendo una lista di String, di modo da far capire chiaramente all'utente i motivi del fallimento.

★ Stats

L'entità **Stats** gestisce lo stato fisiologico del personaggio, tramite dei valori mantenuti nel range 0-100 e offrendo sia controlli sintetici che flag puntuali. Le soglie di dominio sono centralizzate come costanti private e usate da tutti i metodi. I metodi presenti si dividono in due categorie: quelli che verificano la possibilità di eseguire determinate azioni in base alle soglie correnti dei bisogni e quelli di modifica dello stato del personaggio, i quali garantiscono che i limiti non vengano superati, tramite un metodo ausiliario `clamp()`.

2.2.2. Fabbri

★ MainCharacter

L'entità **MainCharacter** è stata progettata come componente del Model (MVC) e applica il principio della composizione, delegando aspetti specifici a **Stats**, **Outfit** e **Hair** per mantenere alta coesione e isolare le regole di dominio. L'accesso ai valori delle statistiche è centralizzato nella classe **Stats** mentre la progressione (lvl, xp, xpToNext) e il contesto (stanza corrente, oggetti già utilizzati) risiedono in **MainCharacter** in quanto parti dello stato persistente del personaggio, mentre l'aggiornamento periodico delega a *stats.decalay()* come fonte unica di verità.

La classe integra un sistema di gestione delle missioni attraverso tre strutture principali:

- **ongoingQuests** per le quest attive
- **completedQuests** per quelle terminate
- **ItemUsedForQuests** che traccia quali oggetti sono stati utilizzati per ogni quest specifica, prevenendo conteggi duplicati e garantendo la corretta progressione.

Il sistema di livellamento implementa una curva di crescita progressiva attraverso *computeXpToNext()*, che utilizza una formula esponenziale per bilanciare le difficoltà di progressione. Il metodo *addXp()* gestisce automaticamente i level-up multipli quando viene guadagnata una grande quantità di esperienza.

L'interazione con il mondo è mediata attraverso il metodo *pickCurrentRoom()*, che implementa un sistema di validazione dell'accesso alle stanze, verificando i prerequisiti e fornendo messaggi dettagliati sulle ragioni dell'eventuale fallimento dell'accesso.

Le interazioni con gli oggetti seguono il pattern **Command** attraverso **PickItemAction** e **DroplItemAction**, restituendo **ActionResult** per ridurre l'accoppiamento, prevenire incoerenze e semplificare test e manutenzione. La gestione dell'inventario è delegata alla classe **Inventory** con capacità definita, mantenendo la separazione delle responsabilità.

★ Outfit e Hair

L'entità **Outfit** è stata realizzata come enum tipizzato, in quanto l'insieme degli stili di vestiti è finito, semplificando così la validazione. All'interno di **MainCharacter** è utilizzata per composizione, così da separare la personalizzazione statica del personaggio da quella dinamica delle statistiche ma garantendo comunque la possibilità di aggiungere nuove opzioni facilmente. L'API minimale facilita la visualizzazione nella View, mantenendo chiaro il contratto del Model.

L'entità **Hair** è stata implementata come enum e, anche qui, si applica il pattern Value Object immutable, che previene stati non validi e rende l'entità sicura da condividere. In **MainCharacter** viene usata per composizione, mantenendo il personaggio coerente. Questa scelta mantiene il dominio consistente e permette l'aggiunta di nuovi stili senza impatti sul codice client.

★ FoodType

FoodType è modellato come enum arricchito, in linea con **Hair** e **Outfit**: funge da catalogo tipizzato e immutabile che centralizza i metadati dei cibi. In **MainCharacter** e negli oggetti di consumo, le regole leggono questi parametri dall'enum, mantenendo bassa la dipendenza dal caso specifico. L'evoluzione nuovamente tratta semplicemente di aggiungere una costante con i suoi metadati.



★ Controller

Il **Controller** è stato progettato per essere il punto di ingresso di tutti gli input, mettendo in comunicazione il Model con la View e garantendo la sincronizzazione tra la logica di gioco e l'interfaccia utente.

Le principali responsabilità implementate includono:

- **Inizializzazione del mondo di gioco:** la creazione completa dell'ambiente avviene tramite il metodo *initializeWorld()*, istanziando tutte le stanze (camera da letto, cucina, bagno, salotto, ripostiglio e giardino), definendone i collegamenti bidirezionali e posizionando gli **NPC** (Mum, Dad e Brother) nelle rispettive stanze. Viene inoltre configurato il **QuestSystem** registrando tutti gli **NPC** e impostando la stanza iniziale del **MainCharacter**, ovvero la camera da letto.
- **Gestione del Game Loop:** lo stato del gioco viene aggiornato ogni 5 secondi attraverso l'uso di un **ScheduledExecutorService**, invocando il decadimento delle statistiche tramite il metodo *stateDecay* e verificando costantemente le condizioni di sconfitta.
- **Navigazione e gestione delle stanze:** il metodo *changeRoom()* gestisce il cambio di stanza del personaggio, verificando l'accessibilità della stanza e aggiornando la visualizzazione. Ad ogni ingresso in una nuova stanza, il controller mostra automaticamente gli oggetti presenti attraverso *itemsInRoom()* e gestisce le eventuali interazioni con gli **NPC** presenti.
- **Gestione delle azioni del giocatore:** l'esecuzione delle interazioni dell'utente, come l'uso, la raccolta o il rilascio di oggetti, viene centralizzata traducendo i comandi della **View** in modifiche dirette sul **MainCharacter**. Per gli oggetti che richiedono una scelta, il controller gestisce la comunicazione bidirezionale con la View attraverso *handleItemChoice()*, processando l'azione in base alla scelta dell'utente.
- **Sistema di Quest:** il **QuestSystem** viene integrato gestendo l'assegnazione di nuove missioni quando il giocatore entra in una stanza, monitorando il progresso delle quest attive e processando il completamento delle stesse attraverso il metodo *tryTurnIn()*. Ogni fase del ciclo di vita delle missioni è accompagnata dalla visualizzazione dei dialoghi appropriati degli **NPC**.
- **Interazioni con gli NPC:** il metodo *handleNpcInteractions()* coordina tutte le interazioni con i personaggi non giocanti, gestendo i dialoghi iniziali, i dialoghi relativi alle quest in corso e interazioni speciali come il sistema di regali della Mum. Il controller mantiene traccia dello stato e delle relazioni attraverso le affinità, aggiornandole costantemente nella **View**.
- **Processamento dei risultati delle azioni:** gli esiti delle azioni vengono interpretati, propagando i messaggi testuali al log della **View** e innescando l'aggiornamento grafico dei componenti.
- **Controllo del ciclo di vita:** gestisce lo stato di "Game Over" disabilitando i controlli della View e mostrando il dialogo di chiusura con la causa del decesso, garantendo che non vengano processate ulteriori azioni dopo la sconfitta.

2.2.3. Giuseppetti

★ NPC

La classe astratta **NPC** modella l'entità generica di un personaggio non giocabile. Essa incapsula lo stato fondamentale del personaggio, inclusi il nome, la posizione corrente (**Room**), il livello di affinità con il protagonista e la lista delle **Quest** assegnabili. La classe mantiene inoltre un riferimento all'oggetto **House**, essenziale per permettere all'NPC di localizzare oggetti (**GameItem**) situati in altre stanze durante la fase di inizializzazione delle missioni, tramite il metodo helper **findItem**.

Dal punto di vista architetturale, si è scelto di applicare il **Template Method Pattern**: la classe base definisce la struttura comune e i metodi concreti per la gestione dello stato (come **addQuest** e **increaseAffinity**), ma delega alle sottoclassi l'implementazione dei dettagli variabili attraverso metodi astratti. Questi includono la generazione dei dialoghi (**getInitialDialogue**, **getQuestAssignedDialogue**, **getQuestInProgressDialogue** e **getQuestCompletionDialogue**) e la logica di creazione delle missioni (**initializeQuests**).

All'interno della classe è definita la **nested enum QuestDifficulty**, che standardizza le ricompense in termini di punti esperienza (XP) e punti affinità in base a tre livelli di difficoltà (EASY, MEDIUM, HARD), garantendo bilanciamento e coerenza tra le diverse missioni del gioco.

★ Mum, Dad and Brother

Le classi **Mum**, **Dad** e **Brother** rappresentano le implementazioni concrete della classe **NPC**. Ciascuna di esse definisce la personalità del membro della famiglia attraverso l'override dei metodi di dialogo, i quali variano dinamicamente in base al livello di affinità corrente (**affinity**).

Il metodo **initializeQuests** è implementato in ogni sottoclasse per costruire catene di missioni narrative. Utilizzando i Predicates funzionali per le **triggerCondition**, le quest vengono rese disponibili sequenzialmente e vengono assegnate solo nel momento in cui il giocatore si trova nella stanza del NPC, è appena entrato nella stanza e infine la quest precedentemente assegnata dallo stesso NPC è stata completata.

La classe **Mum** Include una logica unica gestita dal metodo **checkGiftInteraction**. Al raggiungimento di un'alta affinità (**HIGH_AFFINITY**), questo NPC è in grado di generare e

donare al giocatore un oggetto speciale ("Videogioco retro"), necessario per il completamento di una quest assegnata da un altro NPC (Brother).

★ Quest

L'entità **Quest** rappresenta un obiettivo assegnato da un NPC al MainCharacter e contiene informazioni quali nome, descrizione testuale, riferimento all'NPC assegnatario, ricompense (punti affinità e esperienza) e una lista di condizioni di completamento e le triggerCondition per l'assegnazione della quest.

Per la costruzione delle istanze si è adottato il Builder Pattern, rendendo così possibile l'aggiunta flessibile di parametri ed evitando l'uso di un costruttore con un numero eccessivo di argomenti.

La logica di attivazione delle Quest è gestita da triggerCondition un BiPredicate<MainCharacter, Room> che definisce le precondizioni necessarie affinché la quest venga assegnata al giocatore.

La logica di completamento invece viene gestita da una lista di oggetti CompletionCondition che rappresentano i requisiti pratici della missione. Il metodo checkCompletion itera su questa lista e marca la quest come completata (completed = true) solo se tutte le condizioni sono soddisfatte simultaneamente.

Quando una quest viene portata a termine il metodo applyReward, aggiorna lo stato del giocatore e l'affinità con l'NPC.

Infine il metodo resetItems, si occupa della finalizzazione logica della quest, delegando alle singole condizioni le operazioni di post-completamento, come la rimozione degli oggetti consegnati dall'inventario. .

★ CompletionCondition

L'interfaccia **CompletionCondition** definisce il contratto fondamentale per la gestione dei requisiti di una missione, disaccoppiando la logica di controllo dalla struttura della Quest. Essa espone tre metodi:

- checkCompletion(MainCharacter character), che verifica se la condizione è soddisfatta in un dato momento.
- onQuestCompleted(MainCharacter character), invocato al termine della quest per applicare eventuali effetti collaterali persistenti.
- isAutoCompletable(), metodo booleano che indica se una determinata condizione permette il completamento automatico o se richiede un'interazione manuale (es. parlare con l'NPC).

★ ItemUsageCondition

Modella le quest che richiedono l'interazione con un oggetto senza che questo venga spostato.

Il metodo `checkCompletion` si basa su un flag di stato del personaggio, la quale ci dice se l'oggetto è stato utilizzato o meno.

Il metodo `onQuestCompleted` è lasciato intenzionalmente vuoto, garantendo che l'oggetto rimanga nella sua posizione originale e disponibile per interazioni future.

Infine il metodo `isAutoCompletable` ritorna il valore `true`.

★ ItemDeliveryCondition

Modella le quest di "raccolta e consegna".

Il metodo `checkCompletion` verifica la presenza fisica dello specifico oggetto, (attraverso il suo nome), all'interno dell'inventario del giocatore.

Il metodo `onQuestCompleted` l'oggetto viene rimosso definitivamente dall'inventario, simulando l'atto della consegna all'NPC e prevenendo usi futuri dello stesso oggetto per altre finalità.

Il metodo `isAutoCompletable()` ritorna `false`, in quanto per completare la quest, il giocatore deve "portare" l'oggetto all'NPC.

★ LevelRequirement

L'entità **LevelRequirement** implementa l'interfaccia `Requirement` per rappresentare un vincolo basato sul livello del `MainCharacter`. La classe contiene un valore `requiredLvl` che specifica il livello minimo necessario, validato nel costruttore per garantire che sia almeno 1, lanciando un `IllegalArgumentException` in caso contrario. Il metodo `isSatisfiedBy(MainCharacter)` confronta il livello corrente del personaggio con quello richiesto, restituendo `true` se il requisito è soddisfatto. Il metodo `getFailureReasons(MainCharacter)` costruisce una lista di stringhe che descrive il motivo del fallimento di una determinata azione, includendo sia il livello richiesto che quello attuale del personaggio.

★ QuestSystem

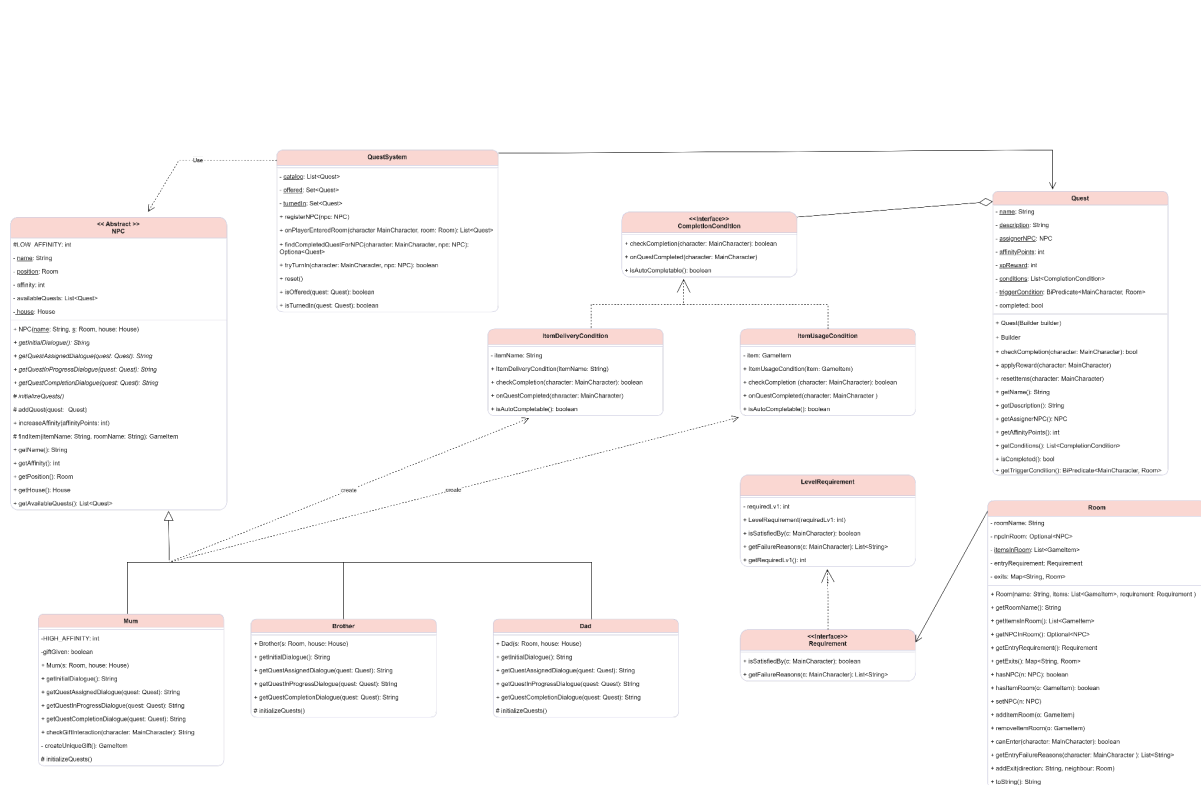
La classe **QuestSystem** gestisce lo stato e l'avanzamento delle missioni in modo reattivo. Essa agisce in risposta agli eventi di gioco (come il cambio di stanza) per aggiornare la disponibilità delle **Quest**. Lo stato delle missioni è affidato a tre strutture dati:

- **Catalog**: una lista completa di tutte le quest disponibili nel gioco, popolata tramite il metodo `registerNPC`, che aggrega le missioni definite nelle singole istanze di NPC.
- **Offered**: un Set che traccia le missioni attualmente attive o proposte al giocatore, garantendo che non vengano offerte duplicazioni.

- **TurnedIn**: un Set che mantiene lo storico delle missioni completate e consegnate, impedendo che vengano riassegnate in futuro.

Il metodo `onPlayerEnteredRoom` viene invocato ogni volta che il giocatore cambia stanza e valuta dinamicamente le condizioni di attivazione di ogni quest presente nel catalogo. Se una condizione è soddisfatta e la quest non è stata ancora affrontata, il sistema la sposta nello stato "offered" e la assegna al personaggio.

Infine, il metodo `tryTurnIn` verifica se il giocatore possiede una quest completata per uno specifico NPC; in caso affermativo, vengono applicate le ricompense (`applyReward`), eseguito il cleanup degli oggetti (`resetItems`) e aggiornato lo stato della missione a `"turnedIn"`.



2.2.4. Neri

★ ActionResult

L'entità **ActionResult** funge da contenitore per l'esito di ogni operazione compiuta nel dominio. Essa permette di trasportare dal **Model** al **Controller** non solo i messaggi da visualizzare (singoli o multipli), ma anche eventuali variazioni di stato, mantenendo le due componenti a basso accoppiamento.

★ Refrigerator

La classe **Refrigerator** rappresenta una specializzazione di **GameItem** dedicata alla gestione di interazioni dinamiche basate sulla scelta dell'utente. A differenza degli oggetti standard, essa funge da **contenitore logico** che mappa diverse opzioni di consumo (**FoodType**) ai rispettivi esiti sulle statistiche del personaggio (**ActionResult**).

Attraverso l'override dei metodi di interazione, la classe implementa un sistema a due fasi:

- **Validazione:** Utilizza il componente **CanEatRequirement** per verificare preventivamente se il **MainCharacter** è in condizione di interagire con il cibo.
- **Risoluzione della Scelta:** Gestisce internamente una **EnumMap** che associa ogni alimento a variazioni specifiche di sazietà, idratazione ed energia.

Questo approccio permette di centralizzare la logica di sostentamento all'interno di un unico oggetto complesso, garantendo che l'interfaccia utente possa interrogare dinamicamente le opzioni disponibili senza conoscere i dettagli implementativi dei singoli cibi.

★ DropItemAction e PickItemAction

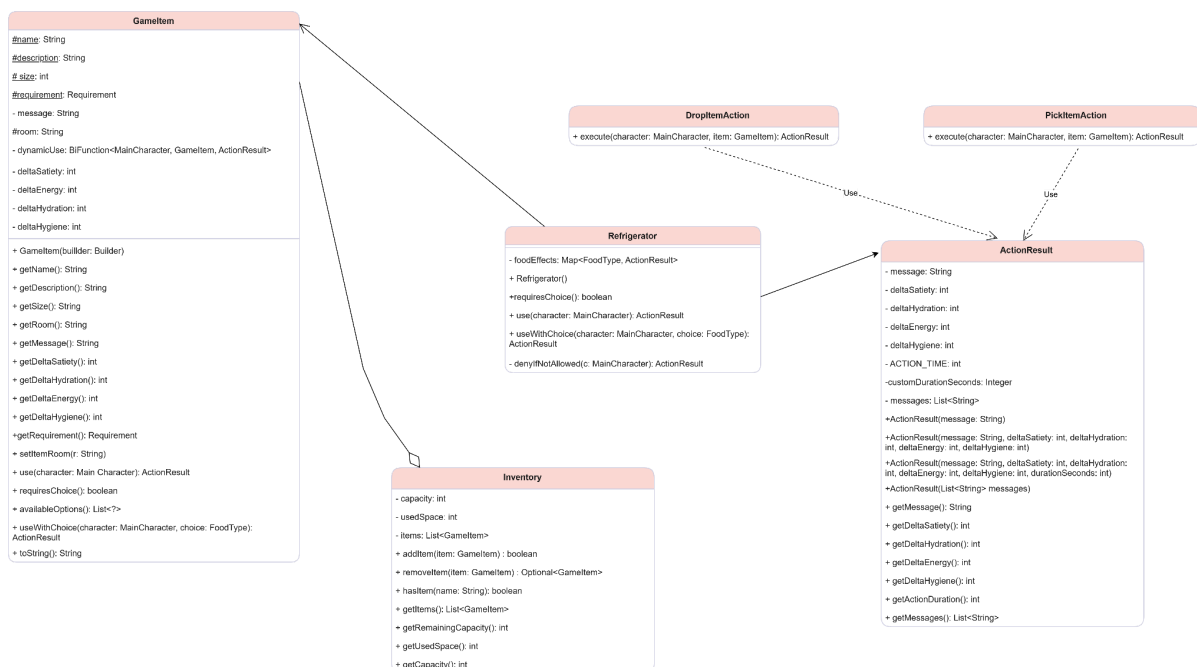
Queste classi modellano le azioni specifiche di interazione con l'inventario. Il **Controller** delega a queste entità la logica di trasferimento degli oggetti tra la stanza corrente e lo zaino del **MainCharacter**, restituendo un **ActionResult** che conferma l'avvenuta operazione o ne spiega il fallimento.

★ Inventory

La classe **Inventory** rappresenta l'inventario del personaggio e si occupa di gestire la collezione di oggetti (**GameItem**) posseduti dal giocatore. A differenza di una semplice lista, l'inventario implementa una logica di **gestione dello spazio limitato**:

- **Gestione della Capacità:** La classe mantiene traccia non solo del numero di oggetti, ma del loro ingombro totale (**usedSpace**) rispetto a una capacità massima definita (**capacity**).

- **Aggiunta Vincolata (addItem):** L'inserimento di un nuovo oggetto è condizionato a una verifica preliminare: il metodo controlla se la somma dello spazio occupato attuale più la dimensione del nuovo oggetto supera la capacità massima. Solo se il controllo ha esito positivo, l'oggetto viene aggiunto e lo spazio occupato aggiornato.
- **Rimozione e Ricerca:** Le operazioni di rimozione (removeItem) e verifica presenza (hasItem) sono implementate sfruttando le **Java Stream API**. Questo permette di cercare gli oggetti per nome in modo dichiarativo ed efficiente. In particolare, la rimozione restituisce un **Optional<GameItem>**, garantendo una gestione sicura del caso in cui l'oggetto non sia presente (evitando **NullPointerException**).
- **Incapsulamento:** Il metodo **getItems()** restituisce una copia immutabile della lista (**List.copyOf**), proteggendo lo stato interno dell'inventario da modifiche non autorizzate provenienti dall'esterno.



★ View

La classe **View** rappresenta il componente centrale della User Interface (UI) e opera come elemento fondamentale del pattern **MVC**. Il suo compito primario è la visualizzazione dello stato del Model e la cattura degli input dell'utente, mantenendo una netta separazione dalla logica di business.

Architettura e Navigazione

Sviluppata tramite la libreria **Java Swing**, la classe estende **JFrame** e adotta un'architettura basata su **CardLayout**. Questa scelta progettuale permette di gestire il ciclo di vita dell'applicazione come una serie di "schermate" distinte, commutabili dinamicamente:

- **MenuPanel**: Schermata iniziale per l'avvio o l'uscita.
- **MainCharacterPanel**: Interfaccia dedicata alla creazione e personalizzazione estetica del protagonista.
- **GamePanel**: L'ambiente principale di gioco dove avviene l'interazione vera e propria.

Organizzazione del GamePanel

Durante la fase di gioco, la View delega la gestione grafica a sub-panel specializzati, organizzati in un layout a tre colonne (o quattro aree principali) per massimizzare l'ergonomia:

- **Monitoraggio Statistiche (StatusPanel)**: Posizionato nella parte superiore, utilizza **JProgressBar** e label testuali per fornire un feedback immediato sui bisogni primari (Energia, Fame, Sete, Igiene), sul livello di esperienza e sulle affinità relazionali con gli NPC.
- **Ambiente Dinamico (RoomPanel)**: Area centrale dedicata alla visualizzazione della stanza corrente. Genera dinamicamente controlli per l'interazione con gli oggetti (**GameItem**) e per la navigazione verso le uscite disponibili.
- **Gestione Risorse (DashboardPanel)**: Pannello laterale che riflette in tempo reale lo stato dell'**Inventory** e l'elenco delle **Quest** attive.
- **Feedback e Narrazione (LogArea)**: Una **JTextArea** non modificabile che funge da console storica, riportando l'esito delle azioni e i dialoghi, garantendo che l'utente riceva sempre un riscontro testuale ad ogni operazione.

Interazione e Comunicazione

La View comunica con il **Controller** tramite il passaggio di eventi e riceve da esso i dati aggiornati tramite oggetti di trasporto come **ActionResult**. L'interazione complessa (come la scelta di un cibo specifico dal frigorifero) è gestita tramite finestre di dialogo modali (**JOptionPane**), che permettono di acquisire scelte dell'utente in modo sincrono senza appesantire il layout principale.

Incapsulamento e Aggiornamento

La classe implementa metodi di aggiornamento specifici (es. `updateStatsDisplay`, `updateInventoryList`) che incapsulano la complessità della UI: il Controller invia dati semplici (stringhe o interi) e la View si occupa di tradurli in elementi grafici (colori delle barre, icone o pulsanti), garantendo un **basso accoppiamento** tra la logica di gioco e la sua rappresentazione.

3. Sviluppo

3.1. Testing automatizzato

3.1.1. HouseTest

L'obiettivo del test è quello di validare le classi **House** e **Room**: si verificherà la corretta creazione e gestione delle stanze, così come la navigazione, la presenza di NPC in determinate stanze, i requisiti di accesso e la gestione dei casi limite.

1. **Creazione e gestione base della casa.** In questo test si crea la casa per poi aggiungere al suo interno 6 stanze predefinite. Si verifica che la casa contenga il numero corretto di stanze e si controlla che tutte le stanze siano presenti con i nomi attesi. Inoltre si testa la navigazione tra stanze e si verifica il comportamento della classe in caso di stanze inesistenti.
2. **Gestione dell'NPC all'interno della stanza.** In questo test si assegnano NPC specifici a stanze diverse e si verifica che ogni stanza riporti correttamente la presenza del suo NPC. Si verifica il comportamento del sistema ad un tentativo di assegnare un NPC duplicato e, infine, si controlla che nelle stanze a cui non sono stati assegnati NPC, non ce ne siano.
3. **Verifica requisiti di accesso alle stanze.** Il test controlla che i requisiti per l'accesso alle stanze funzionino correttamente e che il personaggio possa accedere solo alle stanze concesse dal suo livello. Si aumenta il livello del personaggio e si verifica l'accesso alle stanze con requisito di livello inferiore e dei livelli superiori. Si controllano i messaggi di fallimento per gli accessi negati.
4. **Gestione item nelle stanze.** Si verifica che le stanze gestiscano correttamente gli Item. Si verifica che ogni stanza contenga il numero corretto di item all'inizializzazione, in seguito si aggiunge un item di test a una stanza per verificare l'inserimento corretto e si esegue un'operazione analoga per la rimozione. Infine si testa la verifica di presenza per item non esistenti.
5. **Gestione degli errori.** Si tenta di entrare in una stanza inesistente e si verifica che non venga impostata alcuna stanza corrente. Si tenta di uscire senza stanza corrente impostata e si verifica che venga lanciata l'eccezione appropriata.

3.1.2. GamelItemTest

L'obiettivo di questo test è quello di verificare:

1. **Costruzione corretta via Builder di GamelItem.** Si verifica che i parametri siano stati impostati correttamente e si controlla che i valori non specificati assumano il valore di default.
2. **Azione bloccata se le statistiche del personaggio non rispettano i Requirement.** In questo test si crea un GamelItem con un determinato requisito (CanSleepRequirement) e si tenta di utilizzare l'item con un personaggio che non soddisfa il requisito. Si verifica inoltre che vengano restituiti i messaggi di fallimento appropriati.
3. **Verificare il funzionamento di GamelItem con effetti statici.** Anche in questo caso si crea un GamelItem con un requisito, si modificano le Stats del personaggio per soddisfare il requisito e si utilizza l'Item, verificando il risultato.
4. **Verificare il funzionamento di GamelItem con effetti dinamici.** Si crea un GamelItem con funzione dinamica che modifica l'energia guadagnata in base all'energia corrente e poi si testa l'item con diversi livelli di energia del personaggio. Infine si verifica che gli effetti applicati corrispondano alla logica condizionale.

3.1.3. ItemFactoryKitchenTest

Lo scopo di questo test è quello di validare `ItemFactory.createKitchen()` e il comportamento dei tre item della cucina.

1. **Creazione cucina.** In questo test viene creata la cucina tramite il metodo `ItemFactory.createKitchen()`. Si verifica che il nome della stanza sia "Cucina", si controlla che il requisito di accesso sia un `LevelRequirement` di livello 1 e si conferma che la stanza contenga esattamente 3 item.
2. **Verifica attributi degli item.** Per ogni item all'interno della cucina si verifica che attributi come nome, stanza di appartenenza, dimensione attesa e comportamento rispetto alle scelte sia giusto.
3. **Comportamento di "Fornelli".** Essendo un item con un comportamento dinamico, si verifica il funzionamento con diversi livelli di energia. Inoltre "Fornelli" ha anche un requirement che se non viene soddisfatto non ne permette l'uso.
4. **Comportamento di "Frigorifero".** In questo caso si verifica che il frigorifero richieda la scelta dell'utente, si testa l'interazione base che richiede la selezione di cibo e poi

si testa l'uso con una scelta specifica. Infine si verifica che le statistiche vengano modificate correttamente.

5. **Comportamento di "Lavandino".** Nuovamente per verificare la correttezza dell'azione dinamica.
6. **Requisito di accesso alla cucina.** Si crea un personaggio di livello base e si verifica che soddisfi il requisito di livello 1.

3.1.4. ItemFactoryBedroomTest

In modo analogo al test `ItemFactoryKitchenTest`, qui si cerca di valutare il comportamento di `ItemFactory.createBedroom()` e il comportamento dei **GamelItem** presenti.

1. **Creazione stanza.** Si verifica che la stanza venga creata correttamente, controllando nome, requisito di accesso e numero di item.
2. **Attributi GamelItem.** In questo test si verifica che ogni item nella camera da letto abbia gli attributi corretti.
3. **Funzionamento del letto.** Il letto dispone sia di un requirement che non ne permette l'uso quando il personaggio ha energia elevata, sia di uso dinamico. Questo significa che il guadagno di energia varia in base al livello di energia del personaggio.
4. **Funzionamento del computer.** Il computer viene usato in maniera dinamica e la perdita di energia dipende dal livello di energia del personaggio.
5. **Funzionamento dell'armadio.** Si verifica che gli effetti statici di "Armadio" vengano applicati correttamente.

3.1.5. RefrigeratorTest

Essendo **Refrigerator** un **GamelItem** che estende i comportamenti di un **GamelItem** base, abbiamo deciso di verificarne la validità.

1. **Creazione corretta del frigorifero.** Si verifica che il nome, la stanza e la dimensione siano corretti. Inoltre si verifica che richieda la scelta utente e si controlla che il requisito sia di tipo `CanEatRequirement`.
2. **Offerta delle opzioni alimentari.** Nel secondo test si verifica che il frigorifero offra la lista completa delle opzioni alimentari definite nell'enum `FoodType`.
3. **Messaggio di selezione.** Si verifica che l'uso base del frigorifero restituisca il messaggio di prompt che richiede all'utente di selezionare il cibo. In ogni caso l'uso del frigorifero è possibile solo se si soddisfa il `CanEatRequirement`.

4. **Uso del frigorifero con una scelta alimentare specifica.** Per ogni tipo di cibo nell'enum, si modifica la sazietà del personaggio per soddisfare il requisito, si usa il frigorifero con la scelta del cibo corrente e infine si verifica che messaggio ed effetti corrispondano al cibo selezionato.
5. **Gestione degli errori.** Si verifica il funzionamento del frigorifero in caso di scelta non valida, in questo caso viene restituito un messaggio di errore.
6. **Gestione dei requisiti insoddisfatti.** Se `CanEatRequirement` non è soddisfatto, il frigorifero dovrebbe restituire messaggi d'errore appropriati sia per l'uso base che per l'uso con la scelta.
7. **Gestione dei requisiti soddisfatti.** Si modifica la sazietà del personaggio per soddisfare il requisito e poi si utilizza il frigorifero, prima senza scelta e poi con scelta specifica, verificando messaggio ed effetti.

3.1.6. StatsTest

Lo scopo del test è quello di validare la classe **Stats**. Si verifica che inizializzazione, modifiche, decadimento naturale, stati critici e condizioni di azione funzionino nella maniera corretta.

1. **Inizializzazione e modifiche.** Si verifica che le statistiche siano state inizializzate correttamente a 100, che i metodi di modifica funzionino appropriatamente e che i valori rimangano entro i limiti consentiti. Inoltre si testa il decadimento naturale e gli stati critici.
2. **Condizioni per mangiare.** Si verifica che il metodo `canEat` restituisca true solo quando sono soddisfatte tutte le condizioni: energia > 40, igiene > 50 e sazietà < 80
3. **Condizioni per dormire.** Si verifica che il metodo `canSleep` restituisca true solo quando energia < 70 e igiene > 30;
4. **Condizioni per bere.** Verifica che il metodo `canDrink` restituisca true solo quando idratazione < 70;
5. **Condizioni per lavarsi.** Verifica che il metodo `canShower` restituisca true solo quando igiene < 70 e energia > 20;
6. **Condizioni per giocare.** Verifica che il metodo `canPlay` restituisca true solo quando energia > 20, indipendentemente dalle altre statistiche.

3.2. Metodologia di lavoro

Strumenti utilizzati

- **GitHub:** utilizzato come repository principale;
- **Git:** utilizzato per il versionamento e lo sviluppo cooperativo;
- **Eclipse:** utilizzato come IDE;
- **draw.io:** utilizzato per la creazione e la modellazione dell'UML;
- **Google Documenti:** utilizzato per la redazione della relazione.

3.3. Sorgenti

Link GitHub: https://github.com/nissan345/progetto_pmo_202425.git